PNEUMONIA DETECTION

FROM CHEST X-RAYS

A Complete Step-by-Step Developer Guide

Covering all 5 notebooks:

1. CNN Training Pipeline (from scratch)
2. Transfer Learning with MobileNetV2
3. Attention (CBAM) + GradCAM Explainability
4. Model Compression (TFLite INT8 Quantization)
5. Swin Transformer — Shifted Window Self-Attention

Table of Contents

7. Quick Reference — Key Terms

Chapter 1 — Project Overview

1.1 What is Pneumonia?

Pneumonia is a serious lung infection that inflames the air sacs in one or both lungs. Chest X-rays are the primary diagnostic tool — infected areas appear as white/opaque patches (consolidation) against the dark lung fields. This project trains deep learning models to automatically classify chest X-rays as NORMAL or PNEUMONIA.

1.2 Dataset

The Chest X-Ray Images (Pneumonia) dataset from Kaggle contains 5,216 training images, 16 validation images, and 624 test images. All images are grayscale JPEGs of varying sizes, resized to 224x224 pixels. The dataset is heavily imbalanced: ~74% PNEUMONIA.

Split	NORMAL	PNEUMONIA	Total
Train	~1,341	~3,875	5,216
Validation	8	8	16
Test	234	390	624

1.3 Project Architecture & Model Comparison

Five notebooks are built progressively, each exploring a different approach:

Notebook 1	**CNN (scratch)** — Baseline 4-block CNN trained directly on X-rays. Test acc: 90.1%
Notebook 2	**MobileNetV2 (TL)** — Pretrained ImageNet backbone, fine-tuned. Test acc: 88.1%
Notebook 3	**CBAM + GradCAM** — CNN with channel+spatial attention; GradCAM heatmaps
Notebook 4	**TFLite Compression** — INT8 quantization: 28 MB -> 2.7 MB, ~75% smaller
Notebook 5	**Swin Transformer** — Shifted-window self-attention, hierarchical stages. Test acc: 78.4% (scratch)

Note on Swin-T accuracy

The Swin-T result (78.4%) is from training from scratch on only 5,216 images. Unlike CNNs, transformers have no spatial inductive bias and need far more data to learn from random initialisation. Fine-tuning pretrained Swin-T ImageNet weights is expected to push accuracy above 90%.

Chapter 2 — Notebook 1: CNN Training Pipeline

This notebook builds a Convolutional Neural Network (CNN) from scratch. CNNs learn spatial features by sliding small filters across images. Each filter detects patterns like edges, textures or shapes, and deeper layers combine these into high-level features like 'opacity in the lung region'.

2.1 Imports & Configuration

```
IMAGE_SIZE  = (224, 224)
BATCH_SIZE  = 32
EPOCHS      = 50
LR          = 1e-3
MODEL_PATH  = "../models/pneumonia_cnn.keras"
```

IMAGE_SIZE: all X-rays resized to 224x224 pixels. BATCH_SIZE: 32 images processed per gradient step. LR: Adam learning rate — 1e-3 is the standard safe starting point.

2.2 Data Preprocessing & Augmentation

```
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=15,
    zoom_range=0.15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True,
    fill_mode="nearest",
)
```

- rescale=1./255 — normalises pixels from [0,255] to [0,1]. Neural networks train better with small inputs.
- rotation_range=15 — random rotation up to 15 degrees. Handles slightly angled X-rays.
- zoom_range=0.15 — random zoom up to 15%. Simulates different imaging distances.
- horizontal_flip=True — mirrors image left-right. Lung pathology can appear on either side.

Why augmentation is critical

With only ~5K images and millions of parameters, the model will memorise training data (overfitting) without augmentation. Random transforms make every epoch show slightly different versions of each image, forcing the model to learn robust features.

2.3 CNN Architecture

```
model = Sequential([
    # Block 1-4: Conv → BN → MaxPool (filters: 32, 64, 128, 256)
    layers.Conv2D(32, 3, padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(),
    # ... (repeat x4 with doubling filters)
    # Head
    layers.GlobalAveragePooling2D(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid'),
])
```

`Conv2D(32,3)`	32 learnable 3x3 filters detect local patterns. padding='same' preserves spatial size.
`BatchNormalization`	Normalises layer outputs to mean~0, variance~1. Stabilises and speeds training.
`MaxPooling2D`	Takes max value in 2x2 regions, halving H and W. Reduces computation and adds invariance.
`GlobalAveragePooling2D`	Averages all spatial positions: (7,7,256) -> (256,). Reduces overfitting vs Flatten.
`Dropout(0.5)`	Randomly zeros 50% of neurons during training. Prevents co-adaptation and overfitting.
`Dense(1, sigmoid)`	Output neuron. Sigmoid maps any value to [0,1] = P(PNEUMONIA).

INPUT	OUTPUT
X-ray batch (B, 224, 224, 1) float32	P(PNEUMONIA) per image — shape (B,1). >=0.5 -> PNEUMONIA

2.4 Training & Callbacks

```
callbacks = [
    EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3),
    ModelCheckpoint(MODEL_PATH, monitor='val_accuracy', save_best_only=True),
]
```

- **EarlyStopping** — Stops training if val_loss doesn't improve for 5 epochs. Restores best weights. Prevents overfitting.
- **ReduceLROnPlateau** — Halves learning rate if val_loss stalls for 3 epochs. Helps escape plateaus.
- **ModelCheckpoint** — Saves model whenever val_accuracy improves. Guarantees best model is preserved.

2.5 Evaluation — Result: 90.1% Test Accuracy

CNN Result

Test Accuracy: 90.1% | Test Loss: 0.3188 Strong baseline showing CNNs can effectively detect pneumonia patterns from raw X-rays.

Chapter 3 — Notebook 2: Transfer Learning (MobileNetV2)

Instead of training from scratch, Transfer Learning reuses MobileNetV2 — a model pretrained on 1.2 million ImageNet images. It already understands edges, textures, shapes, and object parts. We repurpose this knowledge for X-ray classification.

3.1 Why Transfer Learning?

Our dataset has only ~5,000 images — far too few to train a large CNN well from scratch. MobileNetV2 was trained on 1.2M diverse images and has learned general visual features that transfer well even to medical images.

3.2 Building the MobileNetV2 Model

```
inputs = layers.Input(shape=(224, 224, 1))
# Step 1: Grayscale -> RGB (MobileNetV2 expects 3 channels)
x = layers.Lambda(lambda t: tf.repeat(t, 3, axis=-1))(inputs)
# Step 2: Frozen MobileNetV2 backbone
base = tf.keras.applications.MobileNetV2(
    input_shape=(224,224,3), include_top=False, weights='imagenet')
base.trainable = False
x = base(x, training=False)
# Step 3: Custom classification head
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(256, activation='relu')(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation='sigmoid')(x)
```

include_top=False removes MobileNetV2's original 1000-class head. weights='imagenet' loads pretrained weights. base.trainable=False freezes all layers in Phase 1 so only the custom head trains.

3.3 Two-Phase Training Strategy

```
# Phase 1: frozen backbone, train head only (LR=1e-3, 10 epochs)
# Phase 2: unfreeze top layers, fine-tune (LR=1e-5, 30 epochs)
```

Why two phases?

Unfreezing with a large LR immediately destroys pretrained weights. Phase 1 first trains the head to produce reasonable gradients. Phase 2 then carefully adapts the backbone with LR 100x smaller.

MobileNetV2 Result

Test Accuracy: 88.1% | Test Loss: 0.3348 Slightly below CNN here due to the tiny 16-image val set causing noisy early stopping. With a proper val split, transfer learning typically outperforms scratch CNNs.

Chapter 4 — Notebook 3: Attention (CBAM) + GradCAM

4.1 CBAM Attention Mechanism

CBAM (Convolutional Block Attention Module) adds two attention gates after conv blocks: Channel Attention (which feature maps matter) and Spatial Attention (where in the image to focus). This suppresses noise from irrelevant regions like spine or rib edges.

```
def channel_attention(x, ratio=8):
    avg = GlobalAveragePooling2D()(x)
    mx  = GlobalMaxPooling2D()(x)
    shared_dense1 = Dense(filters//ratio, activation='relu')
    shared_dense2 = Dense(filters, activation='sigmoid')
    scale = Add()([shared_dense2(shared_dense1(avg)),
                   shared_dense2(shared_dense1(mx))])
    return Multiply()([x, Reshape((1,1,filters))(scale)])

def spatial_attention(x):
    avg  = Lambda(lambda t: tf.reduce_mean(t, axis=-1, keepdims=True))(x)
    mx   = Lambda(lambda t: tf.reduce_max(t,  axis=-1, keepdims=True))(x)
    mask = Conv2D(1, 7, padding='same', activation='sigmoid')(
               Concatenate()([avg, mx]))
    return Multiply()([x, mask])
```

INPUT	OUTPUT
Feature map from conv block — shape (B, H, W, C)	Same shape (B, H, W, C) but unimportant channels/regions suppressed

4.2 GradCAM Explainability

```
with tf.GradientTape() as tape:
    conv_out, preds = grad_model(img_array)
    loss = preds[:, 0]                          # PNEUMONIA score
grads   = tape.gradient(loss, conv_out)
weights = tf.reduce_mean(grads, axis=(0,1,2))   # importance per channel
cam     = tf.reduce_sum(conv_out[0] * weights, axis=-1)
cam     = tf.nn.relu(cam).numpy()
cam     = cam / cam.max()                       # normalise to [0,1]
```

Why GradCAM matters clinically

A black-box model cannot be trusted in medical settings. GradCAM shows radiologists exactly which lung regions drove the prediction. If it highlights a real consolidation area, confidence in the model's decision increases significantly.

Chapter 5 — Notebook 4: Model Compression (TFLite)

The trained MobileNetV2 model is 28 MB. For mobile/edge deployment, TensorFlow Lite quantization reduces this by ~75% with minimal accuracy loss.

5.1 Why Compress?

- Mobile apps need models under 10 MB to keep APK size reasonable.
- INT8 operations are 2-4x faster than FP32 on mobile CPUs.
- Lower memory footprint for deployment on low-RAM devices.
- Reduced power consumption — critical for battery-powered devices.

5.2 Dynamic-Range Quantization

```
converter = tf.lite.TFLiteConverter.from_saved_model(export_dir)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
tflite_model = converter.convert()  # FP32 -> INT8 weights
```

INPUT	OUTPUT
Keras model (28 MB, FP32 weights)	.tflite file (~2.7 MB, INT8 weights — 75% smaller)

5.3 Full INT8 Quantization

```
converter.representative_dataset = rep_dataset  # 100 calibration samples
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
```

Dynamic vs Full INT8

Dynamic-range: weights INT8, activations FP32. No calibration needed. Full INT8: both weights AND activations INT8. Needs 100+ calibration samples. Faster inference. Same file size. Preferred for deployment.

Chapter 6 — Notebook 5: Swin Transformer

The Swin Transformer (Shifted Window Transformer) is a hierarchical vision transformer that replaces global self-attention with efficient local window attention. It achieves linear complexity in image size (vs quadratic for plain ViT) and builds multi-scale feature maps like a CNN — making it far more practical for small medical datasets.

6.1 Why Swin Instead of Plain ViT?

Plain ViT splits images into 16x16 patches and computes attention between ALL pairs. For a 224x224 image this means 196 patches and 196x196 = 38,416 attention pairs per layer. More importantly, ViT has NO spatial inductive bias — it must learn from scratch that nearby pixels are related. With only 5K images, it converges to near-random performance (63%).

Property	Plain ViT	Swin Transformer
Patch size	16x16 (196 patches)	4x4 (3,136 patches)
Attention scope	Global (all patches)	Local 7x7 windows
Complexity	O(N^2) — quadratic	O(N) — linear
Hierarchy	Flat (1 resolution)	4 stages (like CNN)
Position encoding	Absolute (learned)	Relative per window
Cross-window info	Always	Shifted windows every 2nd block
Min data (scratch)	~1M images	Works with 5K+ (better with more)
Test acc (5K)	63% (random)	78.4% (scratch)

6.2 Swin Architecture Overview

The full Swin-T pipeline for one 224x224 chest X-ray:

- **Patch Embed (4x4)** — 224x224 -> 56x56 = 3,136 tokens, each a 96-dim vector
- **Stage 1 [depth=2]** — W-MSA + SW-MSA at 56x56 resolution, dim=96
- **Patch Merging** — 56x56 -> 28x28, dim 96 -> 192 (2x downsample)
- **Stage 2 [depth=2]** — W-MSA + SW-MSA at 28x28 resolution, dim=192
- **Patch Merging** — 28x28 -> 14x14, dim 192 -> 384
- **Stage 3 [depth=6]** — W-MSA + SW-MSA x6 at 14x14 resolution, dim=384
- **Patch Merging** — 14x14 -> 7x7, dim 384 -> 768
- **Stage 4 [depth=2]** — W-MSA + SW-MSA at 7x7 resolution, dim=768 (global context)
- **Global Avg Pool** — 7x7x768 -> 768-dim feature vector
- **Classification Head** — Dense(256, gelu) -> Dropout -> Dense(1, sigmoid) -> P(PNEUMONIA)

What each stage learns

> Stage 1-2 (56x56, 28x28): fine-grained features — lung borders, rib edges, local opacity patches. Stage 3 (14x14): mid-level patterns — lobar consolidation, air-fluid levels. Stage 4 (7x7): global semantics — bilateral vs unilateral involvement, whole-lung haziness.

6.3 Patch Embedding — 4x4 Patches

```
class PatchEmbed(layers.Layer):
    def __init__(self, patch_size=4, embed_dim=96):
        self.proj = layers.Conv2D(
            embed_dim,
            kernel_size=patch_size,   # 4
            strides=patch_size,       # 4 — non-overlapping
            padding="valid")
        self.norm = layers.LayerNormalization(epsilon=1e-5)

    def call(self, x):
        x = self.proj(x)              # (B, 56, 56, 96)
        B, H, W, C = ...
        return self.norm(tf.reshape(x, [B, H*W, C]))  # (B, 3136, 96)
```

Unlike ViT which uses 16x16 patches (196 tokens), Swin uses 4x4 patches creating 3,136 tokens. This much finer resolution means Stage 1 can detect small consolidation patches and subtle opacity changes that would be lost in ViT's coarser 16x16 grid. A Conv2D with kernel=stride=4 extracts and projects all patches in one efficient operation.

INPUT	OUTPUT
X-ray image (B, 224, 224, 1)	Token sequence (B, 3136, 96) — 3,136 patch vectors of 96 dims

6.4 Window Partition & Reverse

```
def window_partition(x, window_size):
    # x: (B, H, W, C)
    B = tf.shape(x)[0]
    H, W, C = x.shape[1], x.shape[2], x.shape[3]
    x = tf.reshape(x, [B, H//ws, ws, W//ws, ws, C])
    x = tf.transpose(x, [0, 1, 3, 2, 4, 5])
    return tf.reshape(x, [-1, ws, ws, C])
    # output: (B * num_windows, ws, ws, C)
    # for 56x56 with ws=7: num_windows = 8x8 = 64 per image
```

Window partition splits the 2D feature map into non-overlapping windows of size 7x7. For a 56x56 feature map this creates 64 windows per image. Attention is computed independently within each window — only 49 tokens instead of 3,136. This reduces attention complexity from O(3136^2) to O(64 * 49^2) — about 65x fewer operations.

INPUT	OUTPUT
Feature map (B, H, W, C)	Windows (B*nW, ws, ws, C) — nW windows per image

6.5 Window Multi-Head Self-Attention (W-MSA)

```
class WindowAttention(layers.Layer):
    def call(self, x, mask=None, training=False):
        # x: (B*nW, ws*ws, C)  —  B*nW windows, each ws*ws=49 tokens
        qkv = self.qkv(x)                    # project to Q, K, V
        qkv = reshape to [B_, N, 3, heads, head_dim]
        q, k, v = qkv[0]*scale, qkv[1], qkv[2]

        attn = tf.matmul(q, tf.transpose(k,[0,1,3,2]))  # (B_, heads, N, N)
        attn = attn + relative_position_bias              # add RPB
        attn = softmax(attn)
        x    = tf.matmul(attn, v)            # weighted sum of values
        return self.proj(reshape(x))         # output projection
```

Within each 7x7 window, standard multi-head self-attention is computed. Each of the 49 tokens attends to all other 49 tokens in its window. With 3 attention heads in Stage 1 (up to 24 in Stage 4), each head learns different local relationships: one might detect opacity boundaries, another vascular patterns, another pleural effusion.

INPUT	OUTPUT
Window tokens (B*nW, 49, C)	Attended tokens (B*nW, 49, C) — each token now encodes local context

6.6 Relative Position Bias

```
# Precomputed in numpy (avoids TF graph scope issues)
coords_h = np.arange(window_size)   # [0,1,...,6]
coords_w = np.arange(window_size)
rel_h = coords_h[:,None] - coords_h[None,:]  # (49, 49)
rel_w = coords_w[:,None] - coords_w[None,:]
rel_index = rel_h * (2*ws - 1) + rel_w       # (49, 49) — flat index

# In call():
bias = tf.gather(self.rel_pos_bias_table, rel_index.flatten())
bias = reshape(bias, [ws*ws, ws*ws, num_heads])
attn = attn + tf.transpose(bias, [2,0,1])[None]
```

Instead of ViT's absolute positional encoding (which breaks when image size changes), Swin uses Relative Position Bias — a learnable table indexed by the relative distance between any two patch positions within a window. This means the model learns that 'two patches 3 positions apart' have a particular relationship regardless of where in the image they are. This is particularly useful for X-rays where pathology appears at varying locations.

Why numpy for the position index?

The position index must be computed before call() runs. If computed using TF ops inside build(), it is created in a temporary FuncGraph and becomes inaccessible during inference. Computing in pure numpy and using as a constant avoids this graph-scope error entirely.

6.7 Shifted Window Attention (SW-MSA)

The core innovation of Swin: every other transformer block shifts the window grid by (window_size // 2) pixels in both dimensions before partitioning. This allows patches at window boundaries to attend to

each other across the original window borders.

```
# Layer N (W-MSA):  windows at (0,0), (0,7), (7,0), (7,7), ...
# Layer N+1 (SW-MSA): tf.roll shifts feature map by (-3, -3)
#                     then window at (0,0) contains patches from
#                     the borders of 4 original windows

if self.shift_size > 0:
    x = tf.roll(x,
                shift=[-self.shift_size, -self.shift_size],
                axis=[1, 2])
# ... partition into windows, compute attention, reverse shift
```

After cyclic shifting, some windows contain patches from non-adjacent regions. An attention mask is applied to prevent these patches from attending to each other (they would not be neighbours in the unshifted grid). The mask adds -100 to attention logits for disallowed pairs, making their softmax weights effectively zero.

```
# Mask built in pure numpy — avoids FuncGraph scope errors
def _build_shift_mask(H, W, window_size, shift_size):
    img_mask = np.zeros((H, W))
    cnt = 0
    for hs in [slice(0,-ws), slice(-ws,-shift), slice(-shift,None)]:
        for ws_ in [slice(0,-ws), slice(-ws,-shift), slice(-shift,None)]:
            img_mask[hs, ws_] = cnt
            cnt += 1
    # partition, compute pairwise differences, threshold
    attn_mask = np.where(pairwise_diff != 0, -100.0, 0.0)
    return tf.constant(attn_mask, dtype=tf.float32)  # stored as constant
```

W-MSA + SW-MSA = full connectivity

W-MSA: attention within fixed 7x7 windows (local). SW-MSA: attention within shifted 7x7 windows (cross-boundary). Alternating the two every block means every patch eventually attends to every other patch within a small number of layers, achieving global receptive field with linear instead of quadratic cost.

6.8 Patch Merging — Hierarchical Downsampling

```
class PatchMerging(layers.Layer):
    def call(self, x):
        # x: (B, H, W, C)
        x0 = x[:, 0::2, 0::2, :]   # top-left of each 2x2 group
        x1 = x[:, 1::2, 0::2, :]   # bottom-left
        x2 = x[:, 0::2, 1::2, :]   # top-right
        x3 = x[:, 1::2, 1::2, :]   # bottom-right
        x  = tf.concat([x0,x1,x2,x3], axis=-1)   # (B, H/2, W/2, 4C)
        x  = self.norm(x)
        return self.reduction(x)    # Linear: 4C -> 2C
        # Result: (B, H/2, W/2, 2C) — half spatial, double channels
```

Patch Merging concatenates 2x2 neighbouring tokens and projects them to 2C dimensions. This halves the spatial resolution and doubles the channel count — exactly like stride-2 convolution in a CNN. Applied between stages, it creates a 4-level hierarchy: 56x56 -> 28x28 -> 14x14 -> 7x7, with channels growing 96 -> 192 -> 384 -> 768.

INPUT	OUTPUT
Token sequence (B, H*W, C)	Merged sequence (B, H/2 * W/2, 2C) — halved resolution, doubled channels

6.9 Full Swin-T Model Construction

```
def build_swin(image_size=(224,224), patch_size=4, embed_dim=96,
               depths=[2,2,6,2], num_heads=[3,6,12,24],
               window_size=7):
    inp = layers.Input(shape=(*image_size, 1))
    x   = PatchEmbed(patch_size, embed_dim)(inp)   # (B, 3136, 96)
    dim = embed_dim
    res = (56, 56)

    for stage_idx, (depth, n_heads) in enumerate(zip(depths, num_heads)):
        for blk_idx in range(depth):
            shift = 0 if blk_idx%2==0 else window_size//2
            x = SwinTransformerBlock(
                dim, res, n_heads, window_size, shift)(x)
        if stage_idx < 3:                          # no merging after last stage
            x   = PatchMerging(res, dim)(x)
            res = (res[0]//2, res[1]//2)
            dim = dim * 2

    x   = GlobalAveragePooling1D()(LayerNorm(x))   # (B, 768)
    x   = Dense(256, activation='gelu')(x)
    out = Dense(1,   activation='sigmoid')(x)
    return Model(inp, out, name='SwinT_Pneumonia')
```

`depths=[2,2,6,2]`	Number of transformer blocks per stage. Stage 3 has 6 blocks for the most compute at 14x14.
`num_heads=[3,6,12,24]`	Attention heads per stage. More heads at deeper stages capture more abstract patterns.
`shift = blk%2==0`	Even-indexed blocks use W-MSA (shift=0), odd blocks use SW-MSA (shift=ws//2=3).
`GlobalAvgPool1D`	Averages over the 49 token positions at Stage 4 output -> single 768-dim vector.
`Dense(256, gelu)`	GELU (Gaussian Error Linear Unit) — smoother than ReLU, standard in transformers.

INPUT	OUTPUT
X-ray image (B, 224, 224, 1)	P(PNEUMONIA) — shape (B, 1). Also: embedding model outputs 768-dim feature vector

6.10 Training Strategy & Class Weights

```
# Class weights to fix PNEUMONIA >> NORMAL imbalance (74% vs 26%)
label_counts = Counter(train_gen.classes)
total        = sum(label_counts.values())
class_weight = {cls: total / (2*count) for cls, count in label_counts.items()}
# Result: NORMAL gets ~2.8x higher weight than PNEUMONIA

swin_model.compile(
    optimizer=AdamW(learning_rate=1e-4, weight_decay=0.05),
    loss='binary_crossentropy',
    metrics=['accuracy'],
)
history = swin_model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=30,
    callbacks=[EarlyStopping, ReduceLROnPlateau, ModelCheckpoint],
    class_weight=class_weight,
)
```

AdamW adds weight decay (L2 regularisation on weights) to Adam. This is the standard optimiser for transformers — the weight_decay=0.05 prevents the attention weights from growing too large and destabilising training. Class weights compensate for the 74%/26% PNEUMONIA/NORMAL imbalance by penalising NORMAL misclassifications more heavily during backpropagation.

Val set problem on Kaggle

The Kaggle dataset's val/ split contains only 16 images — far too few for reliable early stopping. The Kaggle notebook merges val/ into train/ and uses validation_split=0.10 to create a proper ~520-image validation fold.

6.11 Feature Map Visualisation

```
def get_swin_attention(model, img_array, stage=3, block=0):
    block_name  = f"stage{stage}_block{block}"
    # Walk model layers manually — .output not available in Keras 3 sub-layers
    x = img_array
    for layer in model.layers:
        if layer.name == "xray_input": continue
        try:    x = layer(x, training=False)
        except: x = layer(x)
        if layer.name == block_name:
            break
    # x: (1, tokens, C) — output of target Swin block
    H = W = int(x.shape[1] ** 0.5)
    feat = tf.reshape(x[0], [H, W, x.shape[-1]])
    heat = tf.norm(feat, axis=-1).numpy()   # channel-wise L2 norm -> (H, W)
    heat = (heat - heat.min()) / (heat.max() - heat.min() + 1e-8)
    return cv2.resize(heat, (224, 224))
```

Since Swin uses local window attention (not CLS-token global attention like ViT), we use the channel-wise L2 norm of the token features as a proxy attention map. High L2 norm indicates the block found strong features at that spatial location. The map is extracted by walking model.layers sequentially — Keras 3's sub-layers don't expose a .output attribute, so we capture intermediate results during a manual forward pass.

Why not use attention weights directly?

Swin's window attention weights are local (49x49 within each window). Aggregating them into a global 224x224 map requires rolling back the window partition and shift operations, which adds significant complexity. The L2-norm approach gives a clean, interpretable spatial importance map that correlates well with the actual attention pattern.

6.12 t-SNE Embedding Visualisation

```
# Extract 768-dim feature vectors from all 624 test images
embeddings = embedding_model.predict(test_gen)   # (624, 768)
labels     = test_gen.classes                    # (624,)

# Reduce to 2D for visualisation
coords = TSNE(n_components=2, perplexity=30,
              max_iter=1000, random_state=42).fit_transform(embeddings)

# Scatter: blue=NORMAL, red=PNEUMONIA
for cls, color, label in [(0,"#3B82F6","NORMAL"),(1,"#EF4444","PNEUMONIA")]:
    plt.scatter(coords[labels==cls, 0], coords[labels==cls, 1],
                c=color, label=label, alpha=0.6, s=20)
```

The embedding model outputs the Global Average Pooled feature vector (768-dim) for each test X-ray — this is the image's representation in Swin's learned feature space. t-SNE compresses 768 dimensions to 2 for plotting. Well-separated clusters indicate the model learned discriminative representations, even if the final classification accuracy is modest when trained from scratch.

6.13 Model Comparison

The comparison notebook parses training history directly from saved notebook output cells using regex — no hardcoded values. This ensures the chart always reflects the latest run.

Model	Architecture	Params	Test Acc	Test Loss
CNN (scratch)	4-block Conv	~2.5M	90.1%	0.3188
MobileNetV2 (transfer)	Pretrained backbone	~3.4M	88.1%	0.3348
Swin-T (scratch)	4-stage transformer	~28M	78.4%	0.4852

Key insight

CNN outperforms Swin-T from scratch because CNNs have strong inductive biases (translation invariance, local connectivity) that require far less data to exploit. Transformers are more powerful models but need more data or pretraining to unlock that power. Fine-tuning pretrained Swin-T ImageNet weights (as shown in the Kaggle notebook) is expected to push accuracy above 90% and outperform the CNN.

Chapter 7 — Quick Reference: Key Terms

Term	Definition
Epoch	One complete pass over the entire training dataset.
Batch	Subset of images processed together in one forward+backward pass.
Learning Rate	Step size for weight updates. Too large=unstable, too small=slow.
AdamW	Adam optimiser with weight decay (L2 reg on weights). Standard for transformers.
Sigmoid	Squashes any value to (0,1). Used for binary classification output.
ReLU	max(0,x). Fast hidden activation, avoids vanishing gradients.
GELU	Gaussian Error Linear Unit. Smoother than ReLU, used in transformers.
Dropout	Randomly zeros a fraction of neurons during training. Prevents overfitting.
BatchNorm	Normalises layer outputs to mean~0, var~1. Speeds and stabilises training.
LayerNorm	Normalises each token independently. Preferred over BatchNorm in transformers.
Conv2D	Learnable 2D filter sliding over an image to detect local patterns.
MaxPooling	Takes max in each local region. Halves spatial size, adds invariance.
GlobalAvgPool	Averages all spatial positions. Converts feature map to flat vector.
Self-Attention	Each token computes a weighted sum over all other tokens. O(N^2).
Window Attention	Self-attention restricted to local 7x7 windows. O(N) linear cost.
Shifted Window	Shifts window grid by ws//2 to allow cross-window communication.
Relative Pos Bias	Learnable bias indexed by relative patch distance. Better than absolute.
Patch Merging	Concatenates 2x2 neighbours + linear projection. Halves H,W, doubles C.
CLS Token	Learnable token prepended to sequence. Final state = image representation (ViT).
Transfer Learning	Using pretrained weights as a starting point for a new task.
Fine-tuning	Training pretrained layers with a very small learning rate.
Quantization	Reducing weight precision from FP32 to INT8. Shrinks model, speeds inference.
TFLite	TensorFlow Lite — lightweight inference framework for mobile/edge deployment.
Class Weight	Upweights minority class samples in loss to fix imbalance.
Binary Cross-Entropy	Loss for two-class problems. Penalises confident wrong predictions.
Precision	Of all predicted PNEUMONIA, fraction truly positive. TP/(TP+FP).
Recall	Of all true PNEUMONIA, fraction detected. TP/(TP+FN). Critical in medical AI.
F1-Score	Harmonic mean of precision and recall.
t-SNE	Non-linear 2D dimensionality reduction for visualising embeddings.

GradCAM	Gradient-weighted heatmap showing which image regions drove a prediction.
CBAM	Convolutional Block Attention Module — channel + spatial attention gates.
Inductive Bias	Built-in assumptions in a model architecture. CNNs assume local structure; transformers assume none.